

Proyecto 2 - Gramatica Formal de Verilang

Juan Camilo Panadero | Sergio Santana

1. Definicion Formal de la Gramatica (Corregida)

Una gramatica formal se define como la 4-tupla $G = (N, \Sigma, S_0, P)$ donde:

N	Conjunto de no-terminales (categorias sintacticas)
Σ	Alfabeto de terminales (simbolos concretos del lenguaje)
S₀	Simbolo inicial, elemento distinguido de N
P	Conjunto de reglas de produccion

1.1 Conjunto de No-Terminales (N)

En Rascal los no-terminales se escriben en **PascalCase**. La lista actualizada es:

```
N = {
Module,
Import,
Declaration,
DefSpace,
DefOperator, Type, AttributeBlock, Attribute, AttributeValue,
DefVar, VarDef,
DefRule, OperatorApp,
DefExpression, Expression,
QuantifiedExpr,
LogicalExpr,
RelExpr, RelOp, Term,
ID, INT, FLOAT, CHAR, STRING
}
```

1.2 Alfabeto de Terminales (Σ)

Palabras reservadas:

```
"defmodule" "using" "defspace" "defoperator" "defrule"
"defexpression" "defvar" "forall" "exists" "defer" "end" "and" "or" "neg" "in"
```

Operadores logicos, relacionales y aritmeticos:

```
"->" "=>" "≡"
"*" "/" "-" "+" "*" "%"
"<" ">" "<=" ">=" "<>" "="
```

Puntuacion:

```
". " ( " )" "[" "]" ":" ", " "' " "$" "Ø"
```

Caracteres base:

```
"a" | "b" | ... | "z" | "A" | "B" | ... | "Z"  
"0" | "1" | "2" | ... | "9"
```

1.3 Símbolo Inicial (S0)

```
S0 = Module
```

2. Reglas de Producción (P) - Notación EBNF

Convenciones: no-terminales como **Barewords capitalizados** | terminales entre "**comillas**" | | alternativa ?
opcional * cero o mas + uno o mas () agrupación

2.1 Programa y Módulo

Un programa Verilang es exactamente un módulo. Los imports van estrictamente después de la definición del módulo y antes de cualquier declaración.

```
Module ::= "defmodule" ID  
Import*  
Declaration*  
"end"  
  
Import ::= "using" ID
```

2.2 Declaraciones

Componentes internos de un módulo. Pueden aparecer en cualquier orden y cantidad.

```
Declaration ::= DefSpace  
| DefOperator  
| DefRule  
| DefExpression  
| DefVar
```

2.3 Espacios (defspace)

Los espacios definen los elementos del discurso del módulo. El operador < define la relación de sub-espacio (herencia de propiedades del espacio mayor). Space definitions are closed by an end.

```
DefSpace ::= "defspace" ID  
( "<" ID )?  
"end"
```

2.4 Operadores (defoperator)

Notacion currying: operadores de N parametros se expresan encadenando N+1 tipos con ->. Type es ahora recursivo para reflejar directamente la composicion.

```
DefOperator ::= "defoperator" ID ":"  
Type  
AttributeBlock?  
"end"  
  
Type ::= ID  
| ID "->" Type
```

2.5 Atributos

Lista de propiedades entre corchetes. Cada atributo puede tener un valor separado por :, que puede ser un identificador o el simbolo vacio $\emptyset$.

```
AttributeBlock ::= "[" Attribute+ "]"  
  
Attribute ::= ID ( ":" AttributeValue )?  
  
AttributeValue ::= ID  
| " $\emptyset$ "
```

2.6 Variables (defvar)

Variables declaradas con su dominio. Multiples variables separadas por coma dentro del mismo defvar.

```
DefVar ::= "defvar"  
VarDef ( "," VarDef )*  
"end"  
  
VarDef ::= ID ":" Type
```

2.7 Reglas (defrule)

Conecta dos aplicaciones de operador con ->. Cada aplicacion va entre parentesis: el operador seguido de sus argumentos en notacion prefija. Permite anidamiento como (negation p).

```
DefRule ::= "defrule"  
OperatorApp  
"->"  
OperatorApp  
"end"  
  
OperatorApp ::= "(" ID Term+ ")"
```

2.8 Expresiones (defexpression)

Formula logica sobre los elementos de discurso del modulo. Puede ir seguida de atributos opcionales.

```
DefExpression ::= "defexpression"
Expression
AttributeBlock?
"end"

Expression ::= QuantifiedExpr
| LogicalExpr
```

2.9 Cuantificadores (forall / exists)

Cuantificacion de primer orden. Soporta anidamiento arbitrario.

```
QuantifiedExpr ::= ( "forall" | "exists" )
ID "in" ID
"."
Expression
```

2.10 Expresiones Logicas

Conectivos logicos binarios y negacion. La precedencia se codifica directamente en la gramatica mediante niveles (de menor a mayor precedencia).

```
LogicalExpr ::= LogicalExpr "≡" LogicalExpr
> LogicalExpr ">" LogicalExpr
> LogicalExpr "or" LogicalExpr
> LogicalExpr "and" LogicalExpr
> "neg" LogicalExpr
| RelExpr
```

2.11 Expresiones Relacionales

Se agrega infixApp para soportar aplicaciones infijas como $x \text{ isIn } S$ presentes en el ejemplo del enunciado.

```
RelExpr ::= Term RelOp Term
| Term "in" Term
| Term ID Term -- aplicacion infija (nuevo)
| Term

RelOp ::= "=" | "<" | ">"
| "<=" | ">=" | "<>"
```

2.12 Terminos

Unidad basica de una expresion. Incluye OperatorApp para soportar anidamiento de aplicaciones dentro de expresiones.

```
Term ::= OperatorApp
      | ID
      | "(" Expression ")"
      | INT
      | FLOAT
      | CHAR
```

2.13 Identificadores

Secuencia de letras, digitos y guiones que comienza con letra. No puede coincidir con ninguna palabra reservada. Case-insensitive segun especificacion.

```
ID ::= letter ( letter | digit | "-" )*
-- No debe ser palabra reservada
-- Mayusculas equivalen a minusculas
```

2.14 Letras y Digitos

```
letter ::= "a" | "b" | ... | "z"
        | "A" | "B" | ... | "Z"
digit  ::= "0" | "1" | "2" | ... | "9"
```

2.15 Literales Basicos

```
INT ::= digit+
FLOAT ::= digit+ "." digit+
CHAR ::= "'" ( letter | digit ) "'"
```

3. Notas de Diseno

- La gramatica es libre de contexto (Tipo 2 en la jerarquia de Chomsky).
- La precedencia de operadores logicos se codifica directamente en la gramatica mediante el operador '>' de Rascal (a diferencia del Proyecto 1 donde se resolvia mediante parentesis explicitos).
- El currying permite modelar operadores de N parametros como composicion de funciones unarias encadenadas con ->. Type es ahora recursivo para reflejarlo directamente.
- OperatorApp incluye sus propios parentesis "(" ID Term+ ")", permitiendo anidamiento directo como (disjunction (negation p) q).
- El operador < en DefSpace define herencia de propiedades (relacion de sub-espacio).
- Se agrega la regla Term ID Term en RelExpr para soportar expresiones infijas como x isIn S presentes en el ejemplo del enunciado.
- La gramatica es modular y extensible segun indica el enunciado.

4. Cambios Respecto al Proyecto 1

La gramática del Proyecto 1 obtuvo 20/20. Los siguientes cambios fueron necesarios para su implementación en Rascal:

No-terminales en PascalCase

En el Proyecto 1 los no-terminales estaban en minúsculas (program, module, spaceDecl, etc.). Rascal requiere PascalCase: ahora son **Module**, **DefSpace**, **DefOperator**, **QuantifiedExpr**, etc.

Eliminación de program; símbolo inicial con start syntax

El no-terminal `program ::= module` se eliminó. En Rascal se marca directamente con **start syntax Module**. El constructor se llama **verilangModule** para evitar conflicto con la keyword `module` de Rascal.

Layout para whitespace y comentarios

La gramática EBNF no definía whitespace ni comentarios. Rascal requiere definirlos explícitamente. Se agregó soporte para comentarios de línea con `#`.

Palabras reservadas como keyword Reserved

Se agregó un bloque **keyword Reserved** que excluye automáticamente las palabras reservadas de los identificadores, incluyendo `and`, `or`, `neg`, `in`.

Type recursivo para currying

En el Proyecto 1: `type ::= identifier` con el currying en `operatorDecl`. En Rascal: `Type ::= ID | ID "->" Type` — Type es recursivo para reflejar directamente la composición de funciones.

Precedencia codificada en la gramática

En el Proyecto 1 la precedencia se resolvía mediante parentesis explícitos. En Rascal se usa el operador `'>'` para definir niveles de precedencia directamente en `LogicalExpr`, eliminando ambigüedades en el parser.

Nueva regla: aplicación infija de operadores

El enunciado incluye expresiones como `x isIn S` donde un identificador actúa como operador infijo. No estaba en el Proyecto 1. Se agregó **Term ID Term** como alternativa en `RelExpr`.

Listas con separador

En el Proyecto 1: `varDefinition ("," varDefinition)*`. En Rascal se usa azúcar sintáctica: `{VarDef ","}+`.

Operadores especiales escapados

Los operadores `<`, `>`, `->`, `=>`, etc. son especiales en Rascal y se escapan con backslash en las reglas de sintaxis concreta: `\<`, `\>`, `\->`, `\=>`.

Campos opcionales como `list[]` en el AST

Los campos opcionales se representan como `list[T]` con 0 o 1 elemento en lugar de `maybe[T]`, siguiendo la convencion de Rascal.
